

Reducing LLM Costs with Semantic Caching

Semantic caching reduces token consumption, lowers latency and improves user experience by reusing responses based on meaning rather than exact wording, making large language model (LLM) applications more efficient and cost-effective.



Bhavana Anant Giri, developer advocate, Redis

Customer support systems illustrate this challenge clearly. Questions such as “How can I get a refund?”, “Can I get my money back?” and “What is your refund policy?” all express the same intent. Treating each as a separate request forces the application to repeatedly invoke the LLM, increasing costs while offering no additional value to the user.

Why traditional caching falls short

A conventional cache operates by matching exact text. If an incoming request precisely matches a previously stored query, the cached response is returned. Otherwise, the application generates a completely new response.

While this approach works well for conventional software applications, it is less effective for generative AI systems because users rarely phrase identical questions in the same way. Two requests may contain entirely different words while seeking the same information. As a result, traditional caching produces very few cache hits in conversational AI applications.

How semantic caching works

A semantic caching workflow begins when a user’s query is received. The application first converts the query into a vector embedding using an embedding model. The embedding is then compared with existing entries stored in the semantic cache through vector similarity search.

If a sufficiently similar query already exists, the cached response

As organisations increasingly deploy applications powered by large language models (LLMs), managing operational costs has become a growing challenge. Every user request consumes tokens and computational resources, and as applications scale, inference costs and response times increase. While each query may appear unique, many users ask similar questions using different words. Semantic caching addresses this problem by identifying queries that have the same meaning and reusing previously generated responses, reducing both latency and LLM usage.

Unlike conventional caching, which relies on exact keyword matches, semantic caching compares the meaning of user queries using vector embeddings. This enables AI applications to respond more quickly to repeated requests while significantly

lowering operational costs.

The growing cost of LLM applications

A common AI workflow begins when a user submits a query, which is then processed by an LLM to generate a response. Although effective, every request requires a complete inference cycle that typically takes between five and ten seconds. As applications move into production and user traffic grows, token consumption increases rapidly, leading to substantially higher operating costs.

Many of these requests, however, are not entirely unique. Approximately 31 per cent of user queries are duplicates, as users often ask the same question using different words. Processing each request independently results in unnecessary token usage while increasing latency and operational expenses.